

vyvlyk02j

November 21, 2025

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
[2]: print("Veri yükleniyor ve hazırlanıyor...")
try:
    data = pd.read_csv('parkinsons_updrs.data')
    print(f"Veri seti boyutu: {data.shape}")
except FileNotFoundError:
    print("HATA: 'parkinsons_updrs.data' dosyası bulunamadı. Lütfen dosyayı
    ↪yüklediğinizden emin olun.")
    exit()
```

Veri yükleniyor ve hazırlanıyor...

Veri seti boyutu: (5875, 22)

```
[3]: target_col = 'total_UPDRS'
drop_cols = ['total_UPDRS', 'motor_UPDRS']

X = data.drop(drop_cols, axis=1) # Girdiler (Ses özellikleri, yaş, cinsiyet, vb.
    ↪)
y = data[target_col]             # Çıktı (Tahmin edilecek skor)
```

```
[4]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42, shuffle=True)

print(f"Eğitim için veri sayısı: {len(X_train)}")
print(f"Test için veri sayısı: {len(X_test)}")
```

Eğitim için veri sayısı: 4700

Test için veri sayısı: 1175

```
[5]: scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[6]: model = XGBRegressor(
    n_estimators=1500,      # Ağaç sayısı (Daha fazla ağaç = daha hassas öğrenme)
    learning_rate=0.05,    # Öğrenme hızı (Düşük hız + çok ağaç = yüksek başarı)
    max_depth=8,          # Ağaç derinliği (Karmaşık ilişkileri yakalamak için)
    subsample=0.8,        # Her ağaçta verinin %80'ini kullan (Ezberlemeyi_
    ↪ önler)
    colsample_bytree=0.8,  # Her ağaçta sütunların %80'ini kullan
    n_jobs=-1,            # İşlemcinin tüm çekirdeklerini kullan (Hız için)
    random_state=42
)

print("\nModel eğitiliyor... (Bu işlem birkaç saniye sürebilir)")
model.fit(X_train_scaled, y_train)
```

Model eğitiliyor... (Bu işlem birkaç saniye sürebilir)

```
[6]: XGBRegressor(base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=0.8, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric=None, feature_types=None,
    feature_weights=None, gamma=None, grow_policy=None,
    importance_type=None, interaction_constraints=None,
    learning_rate=0.05, max_bin=None, max_cat_threshold=None,
    max_cat_to_onehot=None, max_delta_step=None, max_depth=8,
    max_leaves=None, min_child_weight=None, missing=nan,
    monotone_constraints=None, multi_strategy=None, n_estimators=1500,
    n_jobs=-1, num_parallel_tree=None, ...)
```

```
[7]: print("Test seti üzerinde tahmin yapılıyor...")
y_pred = model.predict(X_test_scaled)

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n" + "="*40)
print(" SONUÇLAR (TELEMONITORING BAŞARISI)")
print("="*40)
print(f"Ortalama Kare Hata (MSE)      : {mse:.2f}")
print(f"Ortalama Mutlak Hata (MAE)     : {mae:.2f}")
print(f"R-Kare (R²) Skoru                : {r2:.4f}")
print("="*40)
```

Test seti üzerinde tahmin yapılıyor...

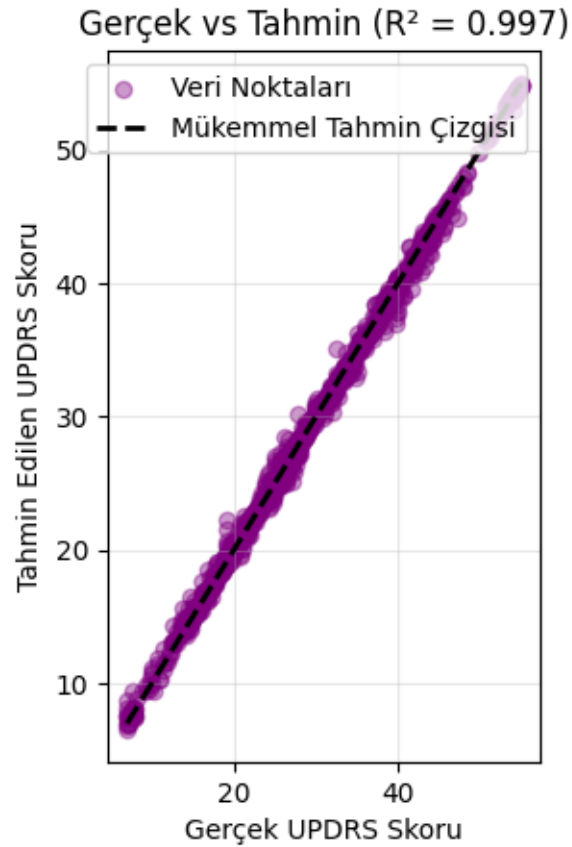
```
=====
SONUÇLAR (TELEMONITORING BAŞARISI)
=====
Ortalama Kare Hata (MSE)      : 0.31
Ortalama Mutlak Hata (MAE)    : 0.37
R-Kare ( $R^2$ ) Skoru          : 0.9972
=====
```

```
[8]: plt.figure(figsize=(12, 6))
```

```
[8]: <Figure size 1200x600 with 0 Axes>
```

```
<Figure size 1200x600 with 0 Axes>
```

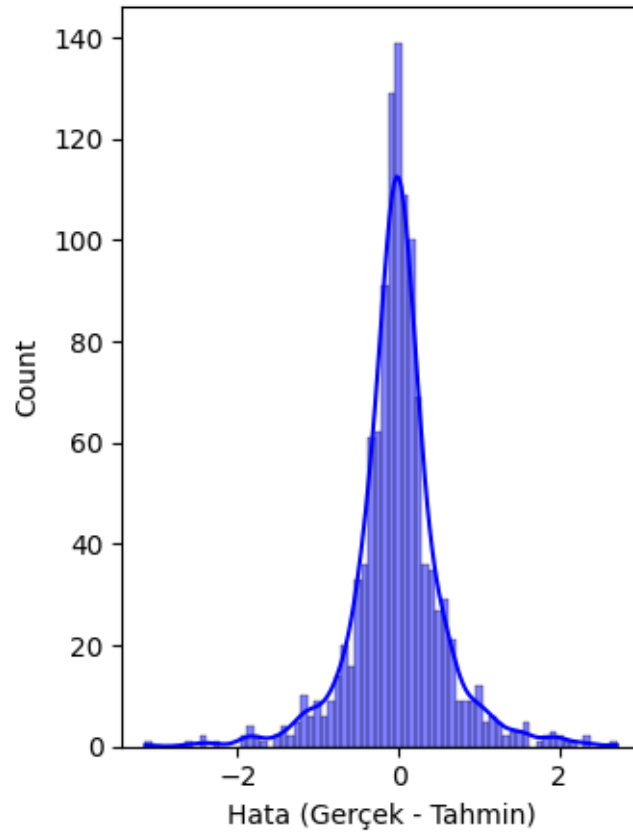
```
[9]: plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred, alpha=0.4, color='purple', label='Veri Noktaları')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=2, label='Mükemmel_
↳Tahmin Çizgisi')
plt.xlabel('Gerçek UPDRS Skoru')
plt.ylabel('Tahmin Edilen UPDRS Skoru')
plt.title(f'Gerçek vs Tahmin ( $R^2 = {r2:.3f}$ )')
plt.legend()
plt.grid(True, alpha=0.3)
```



```
[10]: plt.subplot(1, 2, 2)
residuals = y_test - y_pred
sns.histplot(residuals, kde=True, color='blue')
plt.title('Hata Dağılımı (Sıfıra ne kadar yakın o kadar iyi)')
plt.xlabel('Hata (Gerçek - Tahmin)')

plt.tight_layout()
plt.show()
```

Hata Dağılımı (Sıfıra ne kadar yakın o kadar iyi)



```
[11]: plt.figure(figsize=(10, 6))
sorted_idx = model.feature_importances_.argsort()
plt.barh(X.columns[sorted_idx][-10:], model.
↪feature_importances_[sorted_idx][-10:], color='teal')
plt.title("Model İçin En Önemli 10 Özellik")
plt.xlabel("Önem Derecesi")
plt.show()
```

